

Parallel Loops: Merge Sort

Answer to Question 1

Parallel merge sort implementation and timing: I implemented an iterative bottom up merge sort in mergesort/mergesort.cpp using the OmpLoop helper. To match a parallel loop model, the algorithm avoids recursion. It proceeds in passes of run width 1, 2, 4, and so on. Each pass merges disjoint pairs of runs. To reduce scheduler overhead and improve utilization, the code uses three tactics. First, very small run widths are merged in a single serial pass. Second, for mid size passes there is parallel execution over multiple merges, bundling several merges per loop iteration to coarsen work. Third, when a pass has only a few large merges, it parallelizes inside a merge with a merge path partition that splits the output range among threads. The thread count is set with `setNbThread(nbthreads)` and the OpenMP scheduling chunk size is set with `setGranularity(1)`. Elapsed time is measured with `std::chrono::high_resolution_clock` and printed to `stderr`. Correctness is checked with `checkMergeSortResult` on data produced by `generateMergeSortData`.

Answer to Question 2

Run, plots, and interpretation: I ran `make bench` for the sequential and parallel versions, then produced plots with `make plot`. The trends make sense. For small N, overheads such as thread startup, scheduling, extra passes, and cache effects can outweigh benefits, so speedup may be limited or even below one. For large N, each pass has enough work to amortize overheads and parallel scaling improves. However, bottom up merge sort is memory bandwidth bound, since each pass streams two inputs and writes one output. As thread count grows, bandwidth contention and NUMA effects limit speedup, and efficiency drops. Parallelizing inside the last few large merges helps keep threads busy and improves late pass utilization.

Source: mergesort/mergesort.cpp

```
#include <stdio.h>
#include <iostream>
#include <algorithm>
#include <chrono>
#include <cassert>
#include <climits> // for INT_MIN / INT_MAX
#include "omploop.hpp"

#ifdef __cplusplus
extern "C" {
#endif
    void generateMergeSortData (int* arr, size_t n);
    void checkMergeSortResult (int* arr, size_t n);
#ifdef __cplusplus
}
#endif

// Merge two sorted runs src[l..mid) and src[mid..r) into dst[l..r)
static inline void merge_runs_sequential(const int* src, int* dst,
                                         size_t l, size_t mid, size_t r) {
    if (l >= r) return;
    size_t i = l, j = mid, k = l;
    while (i < mid && j < r) {
        if (src[i] <= src[j]) dst[k++] = src[i++];
        else dst[k++] = src[j++];
    }
    while (i < mid) dst[k++] = src[i++];
    while (j < r) dst[k++] = src[j++];
}

// Given output index K in [0, len), find pair (ia, ib) with ia+ib = K
// using merge-path partitioning (binary search on ia).
static inline void mergepath_partition(const int* A, size_t lenA,
                                       const int* B, size_t lenB,
                                       size_t K, size_t& ia, size_t& ib)
{
    size_t lo = (K > lenB) ? (K - lenB) : 0;
    size_t hi = std::min(K, lenA);
    while (lo < hi) {
        size_t mid = (lo + hi) >> 1;
        size_t j = K - mid; // ib
        // Guard boundaries
        int a_left = (mid > 0) ? A[mid - 1] : INT_MIN;
        int b_left = (j > 0) ? B[j - 1] : INT_MIN;
        int a_right = (mid < lenA) ? A[mid] : INT_MAX;
        int b_right = (j < lenB) ? B[j] : INT_MAX;

        if (a_left > b_right) {
            hi = mid; // decrease ia
        } else if (b_left > a_right) {
            lo = mid + 1; // increase ia
        } else {
            lo = mid; // found
            break;
        }
    }
    ia = lo;
    ib = K - ia;
}
```

Source: mergesort/mergesort.cpp

```
}

// Parallel-in-merge: split output [l, r) into 'chunks' by output index.
// Each chunk merges exactly its slice using precomputed (ia, ib) via merge-path.
static inline void merge_runs_parallel(const int* src, int* dst,
                                      size_t l, size_t mid, size_t r,
                                      OmpLoop& loop, int chunks)
{
    if (l >= r) return;
    if (chunks <= 1) {
        merge_runs_sequential(src, dst, l, mid, r);
        return;
    }

    const int* A = src + l;
    const int* B = src + mid;
    size_t lenA = (mid > l) ? (mid - l) : 0;
    size_t lenB = (r > mid) ? (r - mid) : 0;
    size_t lenOut = lenA + lenB;
    if (lenOut == 0) return;

    size_t tile = (lenOut + static_cast<size_t>(chunks) - 1) / static_cast<size_t>(chunks);
    if (tile == 0) tile = 1;

    // Iterate tiles in output-space [0, lenOut)
    loop.parfor(0, static_cast<int>(lenOut), static_cast<int>(tile), [&](int K0_i) {
        size_t K0 = static_cast<size_t>(K0_i);
        size_t K1 = std::min(K0 + tile, lenOut);

        size_t ia0, ib0;
        mergepath_partition(A, lenA, B, lenB, K0, ia0, ib0);

        size_t i = ia0, j = ib0;
        size_t out = l + K0;

        while (out < l + K1) {
            if (i < lenA && j < lenB) {
                if (A[i] <= B[j]) dst[out++] = A[i++];
                else dst[out++] = B[j++];
            } else if (i < lenA) {
                size_t take = std::min(lenA - i, (l + K1) - out);
                std::copy(A + i, A + i + take, dst + out);
                out += take; i += take;
            } else {
                size_t take = std::min(lenB - j, (l + K1) - out);
                std::copy(B + j, B + j + take, dst + out);
                out += take; j += take;
            }
        }
    });
}

int main (int argc, char* argv[]) {
    if (argc < 3) {
        std::cerr<<"Usage: "<<argv[0]<<" <n> <nthreads>"<<std::endl;
        return -1;
    }
}
```

Source: mergesort/mergesort.cpp

```
const size_t n = static_cast<size_t>(std::max(0, atoi(argv[1])));
const int nbthreads = std::max(1, atoi(argv[2]));

int *a = new int[n];
generateMergeSortData(a, n);
int *b = new int[n];

OmpLoop loop;
loop.setNbThread(nbthreads);
loop.setGranularity(1); // we do our own coarse-graining

// Heuristics
const size_t SERIAL_PASS_THRESHOLD = 64; // runs smaller than this: do pass serially
const int ITER_TARGET_FACTOR = 4; // aim ~ nbthreads*factor iterations per pass

auto t0 = std::chrono::high_resolution_clock::now();

int *src = a;
int *dst = b;

for (size_t width = 1; width < n; width <= 1) {
    const size_t step = (width < 1);
    if (step == 0) break; // overflow guard

    const size_t merges_this_pass = (n + step - 1) / step; // ceil(n/step)

    if (width < SERIAL_PASS_THRESHOLD) {
        // Early passes: tiny runs – do whole pass serially to avoid overhead.
        for (size_t l = 0; l < n; l += step) {
            size_t mid = std::min(l + width, n);
            size_t r = std::min(l + step, n);
            merge_runs_sequential(src, dst, l, mid, r);
        }
    } else if (merges_this_pass >= static_cast<size_t>(nbthreads)) {
        // Plenty of independent merges: parallel over merges with grouping.
        const size_t target_iters = std::max(static_cast<size_t>(nbthreads) *
static_cast<size_t>(ITER_TARGET_FACTOR), static_cast<size_t>(1));
        const size_t group = std::max(static_cast<size_t>(1), (merges_this_pass + target_iters - 1) /
target_iters);
        const size_t outer_inc = step * group;

        loop.parfor(0, static_cast<int>(n), static_cast<int>(outer_inc), [&](int start_i) {
            size_t base = static_cast<size_t>(start_i);
            for (size_t g = 0; g < group; ++g) {
                size_t l = base + g * step; if (l >= n) break;
                size_t mid = std::min(l + width, n);
                size_t r = std::min(l + step, n);
                merge_runs_sequential(src, dst, l, mid, r);
            }
        });
    } else {
        // Too few merges to keep threads busy – parallelize inside each merge.
        size_t denom = std::max(merges_this_pass, static_cast<size_t>(1));
        // compute chunks as size_t then clamp to int >=1
        size_t chunks_sz = static_cast<size_t>(nbthreads) / denom;
        if (chunks_sz < 1) chunks_sz = 1;
        int chunks_per_merge = static_cast<int>(std::min(chunks_sz, static_cast<size_t>(INT_MAX)));
```

Source: mergesort/mergesort.cpp

```
    for (size_t l = 0; l < n; l += step) {
        size_t mid = std::min(l + width, n);
        size_t r    = std::min(l + step,  n);
        merge_runs_parallel(src, dst, l, mid, r, loop, chunks_per_merge);
    }
}

std::swap(src, dst);
}

if (src != a) {
    std::copy(src, src + n, a);
}

auto t1 = std::chrono::high_resolution_clock::now();
std::chrono::duration<double> elapsed = t1 - t0;
std::cerr << elapsed.count() << std::endl;

checkMergeSortResult(a, n);

delete[] a;
delete[] b;
return 0;
}
```







